

UNIVERSITY OF SINDH
INSTITUTE OF MATHEMATICS AND COMPUTER SCIENCE
(IMCS)

INTERNSHIP DEPLOYMENT REPORT

**Detection Engineering, Custom Rule Development,
Log Analysis & Active Response using Wazuh SIEM**

Week 02 – Detection, Custom Rules, Log Analysis & Active Response

Host Organization : Cyberster
Program : Cyberster Blue Team Internship
Department : Blue Team – Security Operations Center (SOC)

Documented By : Junaid Ahmed
Supervisor : Abdullah Zia
Academic Program : BS Computer Science
Track Focus : Cybersecurity | SOC Operations
Submission Date : 06 July 2026

TABLE OF CONTENTS

- Executive Summary
- Week 2 Objectives

1. Task 1 – File Integrity Monitoring (FIM)

- 1.1 File Integrity Monitoring Fundamentals
- 1.2 Syscheck Configuration (`ossec.conf`)
- 1.3 Monitored Directory Configuration
- 1.4 Real-Time FIM Testing Methodology
- 1.5 Alert Investigation & Dashboard Analysis
- 1.6 FIM Event Analysis Report
- 1.7 Key Findings

2. Task 2 – Custom Detection Rule Engineering

- 2.1 Wazuh Rule Engine Architecture
- 2.2 Custom Rule Development (`local_rules.xml`)
- 2.3 Rule 1 – Linux User Account Creation Detection
- 2.4 Rule 2 – SSH Brute Force Detection
- 2.5 Rule 3 – Windows USB Device Detection
- 2.6 Rule Validation & Testing
- 2.7 False Positive Analysis & Rule Tuning
- 2.8 Key Findings

3. Task 3 – Enterprise Security Log Analysis

- 3.1 Linux Authentication Log Analysis
- 3.2 Windows Security Event Log Analysis
- 3.3 Threat Hunting & Event Correlation
- 3.4 Suspicious Activity Timeline
- 3.5 Analytical Findings

4. Task 4 – Active Response Implementation

- 4.1 Active Response Architecture
- 4.2 Active Response Configuration (`ossec.conf`)
- 4.3 SSH Brute Force Simulation
- 4.4 Firewall Verification (`iptables`)
- 4.5 Active Response Workflow Analysis
- 4.6 Risks & Production Considerations

5. Operational Challenges & Troubleshooting

- 5.1 Configuration Challenges
- 5.2 Troubleshooting Process
- 5.3 Solutions Implemented

6. Skills & Knowledge Gained

- Conclusion
- References

Appendices

- Appendix A – Commands Executed

Executive Summary

This report documents the activities completed during Week 2, focusing on the implementation of practical detection capabilities, custom detection rule development, security log analysis, and automated active response using Wazuh SIEM. Building upon the operational environment established in the previous week, the objective was to enhance the platform's ability to detect, analyze, and respond to security events within a controlled laboratory environment.

The first phase involved configuring File Integrity Monitoring (FIM) through the Wazuh Syscheck module. Critical system directories were monitored in real time to detect file creation, modification, and deletion events. The resulting alerts were investigated to understand how Wazuh captures file activity and provides valuable forensic context for security monitoring.

The second phase focused on detection engineering by designing and implementing custom XML-based detection rules. Rules were developed to identify Linux user account creation, SSH brute-force attempts, and Windows USB device insertion events. Each rule was validated through controlled testing and refined to minimize false positives while improving detection accuracy.

The third phase concentrated on security log analysis across Linux and Windows systems. Authentication logs and Windows Security Event Logs were examined to identify suspicious activities, correlate events across multiple hosts, and construct a chronological timeline of security-relevant events. Significant findings were documented and exported for reporting purposes.

The final phase involved configuring and validating Wazuh Active Response to automatically mitigate SSH brute-force attacks using the firewall-drop mechanism. The complete workflow—from attack simulation and alert generation to automated IP blocking and firewall verification—was successfully tested to demonstrate an automated defensive response.

Overall, the activities completed during this week strengthened practical skills in detection engineering, security monitoring, threat analysis, incident investigation, and automated response. The experience provided a deeper understanding of how SIEM technologies support Security Operations Centers (SOCs) in identifying, analyzing, and responding to security threats through proactive monitoring and evidence-driven investigation.

Week 2 Objectives

The primary objective of Week 2 was to enhance the operational capabilities of the Wazuh SIEM environment by implementing advanced detection, monitoring, and response mechanisms. This phase focused on strengthening the ability to identify security events, develop custom detection logic, analyze system logs, and automate defensive actions against potential threats.

The specific objectives of this week's activities were:

- Configure and validate **File Integrity Monitoring (FIM)** using the Wazuh Syscheck module to detect unauthorized file creation, modification, and deletion events.
- Develop and implement **custom XML-based detection rules** to identify Linux user account creation, SSH brute-force attacks, and Windows USB device insertion events.
- Perform **security log analysis** using Linux authentication logs and Windows Security Event Logs to identify suspicious activities, correlate events, and generate meaningful security insights.
- Configure and validate **Wazuh Active Response** to automatically block malicious SSH brute-force attempts using the firewall-drop mechanism.
- Gain practical experience in **detection engineering, security monitoring, threat analysis, incident investigation, and automated response** using enterprise SIEM technologies within a controlled laboratory environment.

1. Task 1 – File Integrity Monitoring (FIM)

1.1 File Integrity Monitoring Fundamentals

File Integrity Monitoring (FIM) is a security control that continuously monitors critical files and directories to detect unauthorized or unexpected changes. It plays a vital role in identifying potential security incidents by tracking file creation, modification, deletion, permission changes, and ownership changes. By comparing the current state of monitored files against a trusted baseline, FIM enables organizations to detect suspicious activities that may indicate system compromise, malware infection, unauthorized configuration changes, or insider threats.

Within Wazuh SIEM, File Integrity Monitoring is implemented through the **Syscheck** module. Syscheck periodically scans configured directories, calculates cryptographic hash values, and compares them with previously recorded values. Whenever a change is detected, Wazuh generates an alert containing detailed information about the affected file, the type of modification, timestamps, user information (when available), and the associated security rule. These alerts assist security analysts in investigating incidents and maintaining the integrity of critical system resources.

In this task, the Syscheck module was configured to monitor selected directories on the target systems. Controlled file creation, modification, and deletion activities were then performed to validate that Wazuh successfully detected each event and generated corresponding alerts within the Wazuh Dashboard. The resulting telemetry was analyzed to evaluate the effectiveness of File Integrity Monitoring in supporting security monitoring and incident investigation.

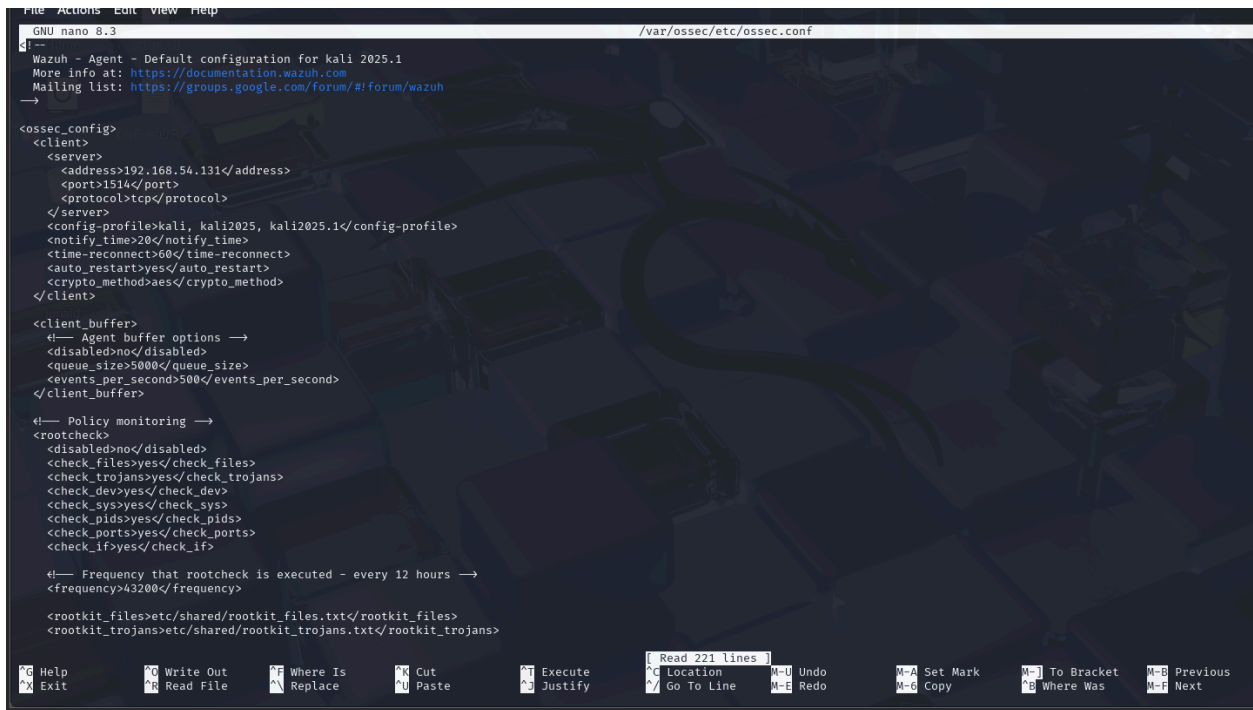
1.2 Syscheck Configuration (`ossec.conf`)

To enable File Integrity Monitoring (FIM), the Syscheck module was configured by modifying the `ossec.conf` file on the Linux Wazuh agent. The objective was to monitor critical system

directories that commonly contain configuration files and security-relevant logs, allowing Wazuh to detect unauthorized file creation, modification, and deletion events in real time.

The Syscheck configuration was updated to monitor the `/etc` and `/var/log` directories. Real-time monitoring (`realtime="yes"`) was enabled for both directories to ensure file system events were detected immediately without waiting for the next scheduled scan. Additionally, the `report_changes="yes"` attribute was enabled for the `/etc` directory to capture the exact content differences whenever monitored text files were modified. This option was intentionally not enabled for `/var/log` because log files change frequently, which would generate excessive and unnecessary event data.

The scheduled scan frequency was also configured to an appropriate interval for the laboratory environment, ensuring periodic verification of monitored files while complementing the real-time monitoring capability. After completing the configuration, the Wazuh agent service was restarted to apply the new settings, and the agent status was verified to confirm successful operation before proceeding with functional testing.



```

GNU nano 8.3 /var/ossec/etc/ossec.conf
Wazuh - Agent - Default configuration for Kali 2025.1
More info at: https://documentation.wazuh.com
Mailing list: https://groups.google.com/forum/#!forum/wazuh

<ossec_config>
<client>
  <server>
    <saddress>192.168.54.131</saddress>
    <port>1514</port>
    <protocol>tcp</protocol>
  </server>
  <config-profile>kali, kali2025, kali2025.1</config-profile>
  <notify_time>20</notify_time>
  <time-reconnect>60</time-reconnect>
  <auto_restart>yes</auto_restart>
  <crypto_method>aes</crypto_method>
</client>

<client_buffer>
  <!-- Agent buffer options -->
  <disabled>no</disabled>
  <queue_size>5000</queue_size>
  <events_per_second>500</events_per_second>
</client_buffer>

<!-- Policy monitoring -->
<rootcheck>
  <disabled>no</disabled>
  <check_files>yes</check_files>
  <check_trojans>yes</check_trojans>
  <check_dev>yes</check_dev>
  <check_sys>yes</check_sys>
  <check_pids>yes</check_pids>
  <check_ports>yes</check_ports>
  <check_if>yes</check_if>

  <!-- Frequency that rootcheck is executed - every 12 hours -->
  <frequency>43200</frequency>

  <rootkit_files>etc/shared/rootkit_files.txt</rootkit_files>
  <rootkit_trojans>etc/shared/rootkit_trojans.txt</rootkit_trojans>

```

Figure 1.1: Opening the Wazuh Agent Configuration File (`ossec.conf`)

The `ossec.conf` configuration file was opened on the Linux Wazuh agent to configure the Syscheck module for File Integrity Monitoring (FIM). This file contains the monitoring settings used by the Wazuh agent.

1.3 Monitored Directory Configuration

System	Directory	Reason
Linux	/etc	Critical configuration files
Linux	/var/log	Security and system logs
Windows	C:\Windows\System32	Core operating system files

```

<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Directories to check (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>

  <!-- Files/directories to ignore -->
  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>
  <ignore>/etc/mail/statistics</ignore>
  <ignore>/etc/random-seed</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/adjtime</ignore>
  <ignore>/etc/httpd/logs</ignore>
  <ignore>/etc/utmpx</ignore>
  <ignore>/etc/wtmpx</ignore>
  <ignore>/etc/cups/certs</ignore>
  <ignore>/etc/dumpdates</ignore>
  <ignore>/etc/svc/volatile</ignore>

  <!-- File types to ignore -->
  <ignore type="sregex">.log$.swp$</ignore>

  <!-- Check the file, but never compute the diff -->
  <nodiff>/etc/ssl/private.key</nodiff>

  <skip_nfs>yes</skip_nfs>
  <skip_dev>yes</skip_dev>
  <skip_proc>yes</skip_proc>
  <skip_sys>yes</skip_sys>

  <!-- Nice value for Syscheck process -->
  <process_priority>10</process_priority>

  <!-- Maximum output throughput -->

```

Figure 1.2: Default Syscheck Configuration on the Linux Wazuh Agent

The `<syscheck>` section of the Wazuh agent configuration file was located to review the existing File Integrity Monitoring settings before applying the required Week 2 configuration changes.

```

<disabled>no</disabled>

<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>600</frequency>

<scan_on_start>yes</scan_on_start>

<!-- Directories to check (perform all possible verifications) -->
<directories realtime="yes" report_changes="yes">/etc</directories>
<directories realtime="yes">/var/log</directories>

```

Figure 1.3: Updated Syscheck Configuration for File Integrity Monitoring

The Syscheck configuration was updated to enable real-time monitoring of the `/etc` and `/var/log` directories. Content change reporting was enabled for `/etc`, and the scan frequency was reduced to 600 seconds to improve monitoring responsiveness in the lab environment.

```

(kali@kali)-[~]
$ sudo systemctl restart wazuh-agent

(kali@kali)-[~]
$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-07-04 18:56:30 EDT; 23s ago
     Invocation: 791ca3fc1777497bb580a5f3e8c8047c
   Process: 12760 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 29 (limit: 2165)
   Memory: 20.4M (peak: 36.6M)
      CPU: 11.033s
   CGroup: /system.slice/wazuh-agent.service
           └─12783 /var/ossec/bin/wazuh-execd
             └─12801 /var/ossec/bin/wazuh-agentd
               └─12822 /var/ossec/bin/wazuh-syscheckd
                 └─12832 /var/ossec/bin/wazuh-logcollector
                   └─12858 /var/ossec/bin/wazuh-modulesd

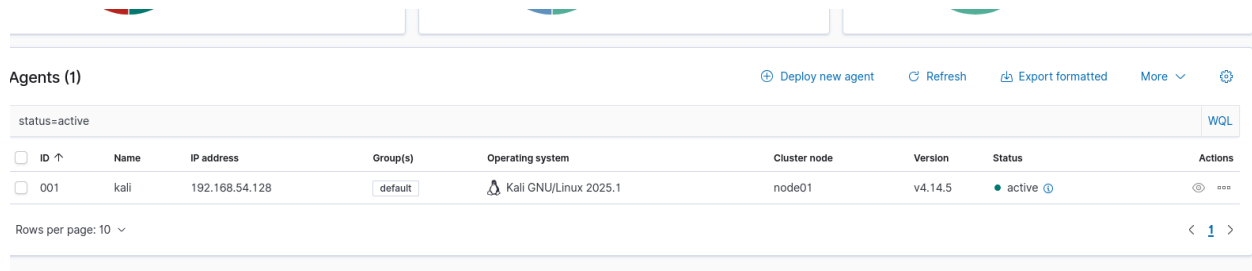
Jul 04 18:56:24 kali systemd[1]: Starting wazuh-agent.service - Wazuh agent ...
Jul 04 18:56:24 kali env[12760]: Starting Wazuh v4.14.5 ...
Jul 04 18:56:25 kali env[12760]: Started wazuh-execd ...
Jul 04 18:56:26 kali env[12760]: Started wazuh-agentd ...
Jul 04 18:56:26 kali env[12760]: Started wazuh-syscheckd ...
Jul 04 18:56:27 kali env[12760]: Started wazuh-logcollector ...
Jul 04 18:56:28 kali env[12760]: Started wazuh-modulesd ...
Jul 04 18:56:30 kali env[12760]: Completed.
Jul 04 18:56:30 kali systemd[1]: Started wazuh-agent.service - Wazuh agent.

(kali@kali)-[~]
$

```

Figure 1.4: Wazuh Agent Restart After Applying FIM Configuration

After updating the File Integrity Monitoring configuration, the Wazuh agent service was restarted to apply the new settings. The service status confirmed that the agent was running successfully with the updated Syscheck configuration.



ID	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
001	kali	192.168.54.128	default	Kali GNU/Linux 2025.1	node01	v4.14.5	active	

Figure 1.5: Linux Wazuh Agent Successfully Connected After Configuration Update

The Wazuh dashboard confirmed that the Linux agent remained active after the configuration changes, indicating that the updated File Integrity Monitoring settings were successfully applied without affecting agent connectivity.

1.4 Real-Time FIM Testing Methodology

For Linux:

- Created file
- Modified file
- Deleted file

Now we'll create a file inside `/etc`.

```
echo "Week 2 FIM Test" | sudo tee /etc/fim_test.txt
```

This creates a new file in `/etc`, which should trigger a File Integrity Monitoring alert.

```
Jul 04 18:56:30 kali systemd[1]: Started wazuh-agent.service - Wazuh agent.
(kali@kali)-[~]
$ echo "Week 2 FIM Test" | sudo tee /etc/fim_test.txt
Week 2 FIM Test
(kali@kali)-[~]
$
```

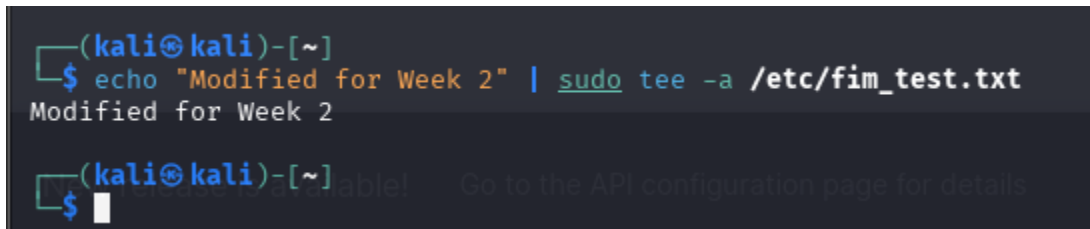
Figure 1.6: Creating a Test File in the Monitored `/etc` Directory

A test file named `fim_test.txt` was created in the monitored `/etc` directory to generate a File Integrity Monitoring (FIM) event. This action was performed to verify that Wazuh detects newly created files in real time.

Modify the File

```
echo "Modified for Week 2" | sudo tee -a /etc/fim_test.txt
```

This appends a new line to the file, generating a **modified** event.



```
(kali㉿kali)-[~]
$ echo "Modified for Week 2" | sudo tee -a /etc/fim_test.txt
Modified for Week 2
(kali㉿kali)-[~]
$
```

Figure 1.7: Modifying the Test File to Trigger a FIM Change Event

The test file was modified by appending additional content, allowing Wazuh to detect and record a file modification event through the Syscheck module.

Delete the File

```
sudo rm /etc/fim_test.txt
```



```
Modified for Week 2
(kali㉿kali)-[~]
$ sudo rm /etc/fim_test.txt
(kali㉿kali)-[~]
$
```

Figure 1.8: Deleting the Test File from the Monitored Directory

The test file was removed from the monitored directory to generate a file deletion event, completing the File Integrity Monitoring validation process.

1.5 Alert Investigation & Dashboard Analysis

	Jul 4, 2026 @ 19:15:11.726	kali	Integrity checksum changed.	7	550
	Jul 4, 2026 @ 19:15:11.362	kali	File deleted.	7	553
	Jul 4, 2026 @ 19:15:01.753	kali	Integrity checksum changed.	7	550

Figure 1.9: File Integrity Monitoring Alerts Generated by Wazuh

Wazuh successfully detected the creation, modification, and deletion of the test file in the monitored `/etc` directory. The alerts confirm that the configured File Integrity Monitoring policy is functioning correctly.

1.6 FIM Event Analysis Report

Event	Agent	Rule ID	Severity	Observation
File Created	Kali	554	5	New file detected
File Modified	Kali	550	7	Integrity checksum changed
File Deleted	Kali	553	7	File removal detected

If these events were observed in a production environment without prior authorization, they would require immediate verification to determine whether the changes were legitimate or malicious. The affected system, user account, and associated processes would be reviewed before deciding on containment or further investigation.

File Path	Event	Rule ID	Agent	Security Analysis
<code>/etc/fim_test.txt</code>	Integrity checksum changed	550	Kali	The alert indicates that the contents of the monitored file were modified. In a production environment, unexpected changes to configuration files should be investigated to determine whether they were authorized.
<code>/etc/fim_test.txt</code>	File deleted	553	Kali	Wazuh detected the removal of a monitored file. Unauthorized deletion of system files could indicate malicious activity or an attempt to remove forensic evidence.

Windows FIM Configuration

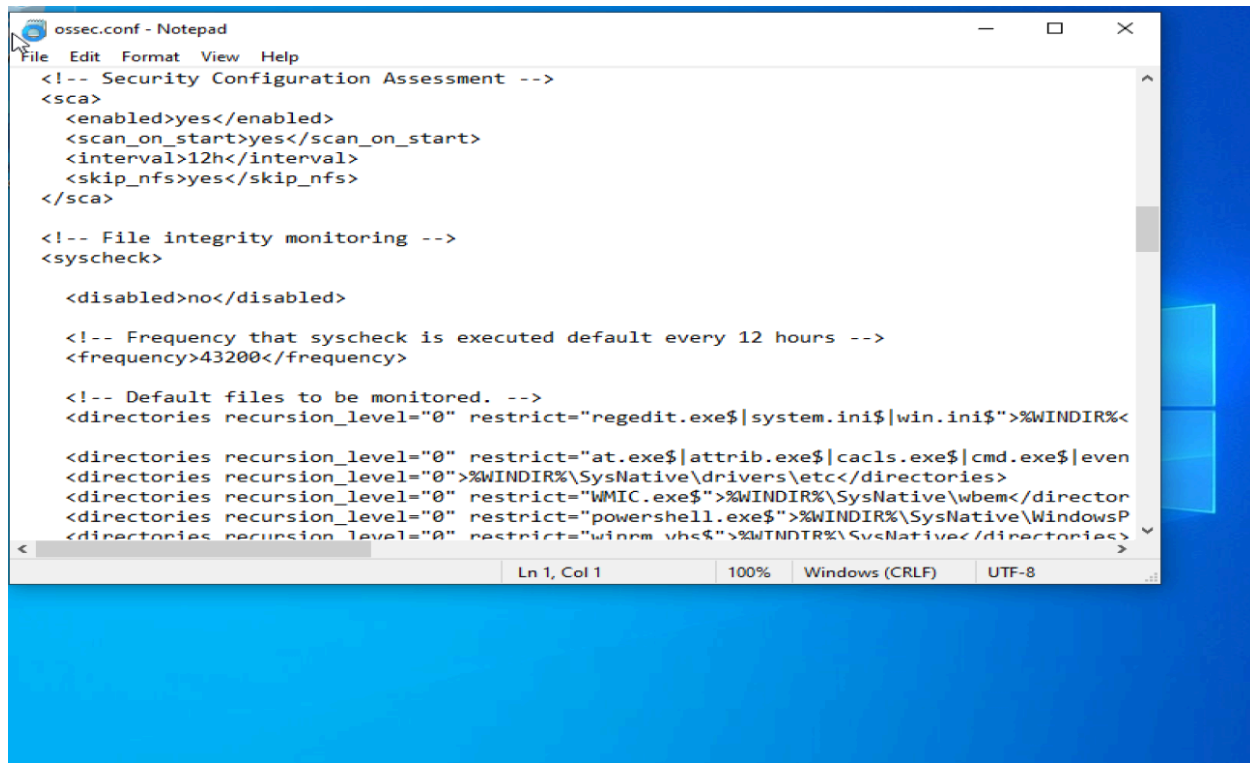


Figure 1.11: Opening the Wazuh Agent Configuration on Windows

The Wazuh agent configuration file was opened on the Windows endpoint to configure File Integrity Monitoring for critical system directories.

```

<disabled>no</disabled>

<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>600</frequency>
<directories realtime="yes" report_changes="yes"> C:\Windows\System32</directories>

<!-- Default files to be monitored. -->
<directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$" %WINDIR%>
<directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|even

```

Figure 1.12: Updated Windows Syscheck Configuration

The Windows Wazuh agent was configured to monitor the C:\Windows\System32 directory using real-time File Integrity Monitoring with content change reporting enabled. The scheduled scan frequency was also reduced to 600 seconds for faster integrity verification.

Restart the Wazuh Agent

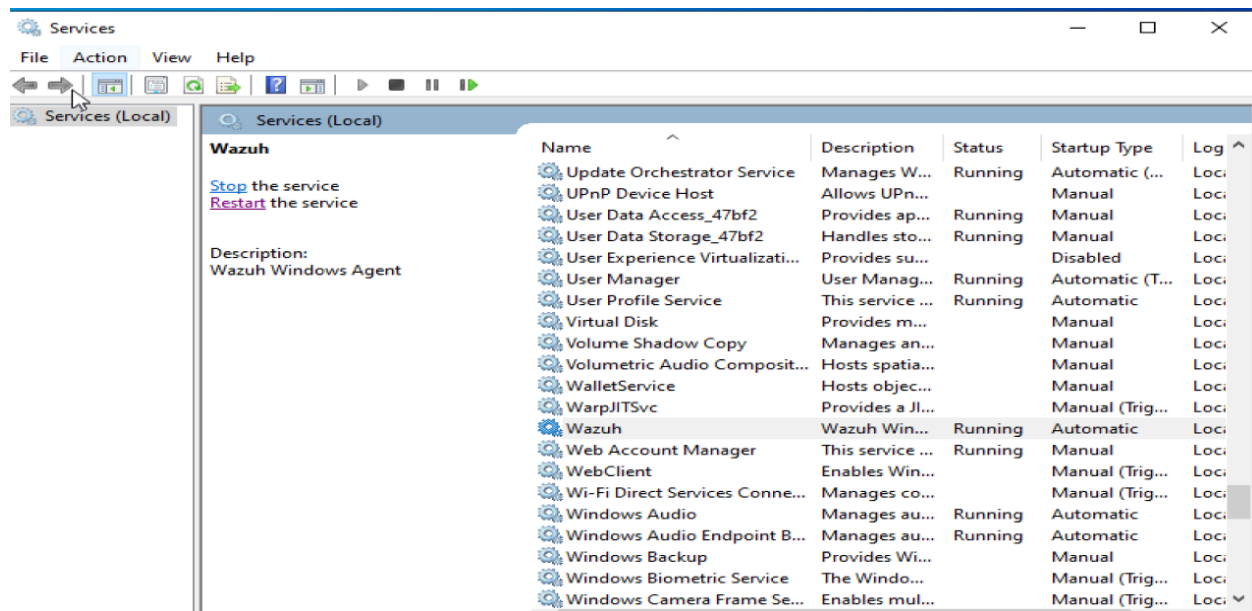


Figure 1.13: Restarting the Windows Wazuh Agent Service

The Wazuh agent service was restarted to apply the updated File Integrity Monitoring configuration. Restarting the service ensures the modified `ossec.conf` settings are loaded and begin monitoring the configured directory.

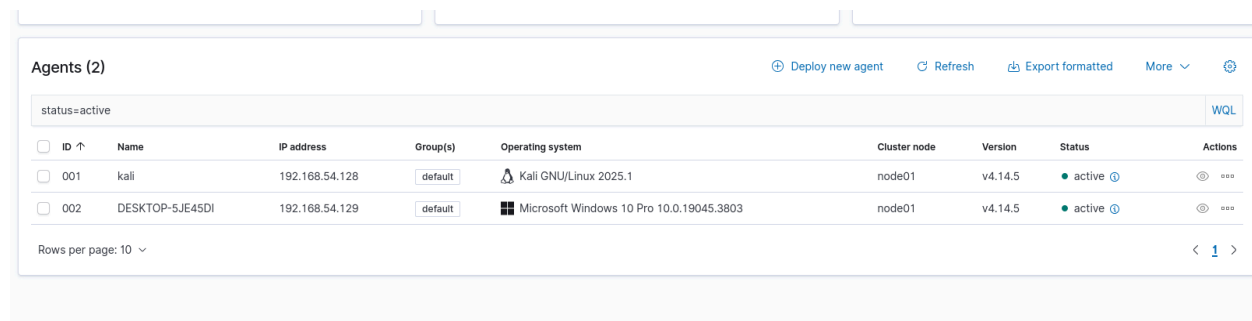


Figure 1.14: Windows Agent Successfully Reconnected

After restarting the service, the Windows endpoint successfully reconnected to the Wazuh Manager, confirming that the updated File Integrity Monitoring configuration was applied successfully.

Windows FIM events (create, modify, and delete a test file in `C:\Windows\System32`) and capture the resulting alerts

Create a Test File

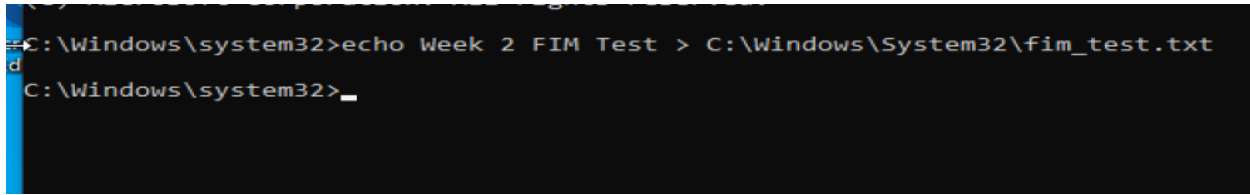


Figure 1.15: Creating a Test File in the Monitored Windows Directory

A test file was created inside the monitored `C:\Windows\System32` directory to generate a File Integrity Monitoring event and verify that the Windows Wazuh agent detected the change.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 4, 2026 @ 19:59:33.723	kali	Integrity checksum changed.	7	550
Jul 4, 2026 @ 19:59:31.003	DESKTOP-SJE45DI	Registry Value Entry Added to the System	5	752
Jul 4, 2026 @ 19:59:30.962	DESKTOP-SJE45DI	Registry Key Integrity Checksum Changed	5	594
Jul 4, 2026 @ 19:59:29.376	DESKTOP-SJE45DI	Registry Value Integrity Checksum Changed	5	750
Jul 4, 2026 @ 19:59:29.376	DESKTOP-SJE45DI	Registry Value Integrity Checksum Changed	5	750
Jul 4, 2026 @ 19:59:29.334	DESKTOP-SJE45DI	Registry Value Integrity Checksum Changed	5	750
Jul 4, 2026 @ 19:59:29.320	DESKTOP-SJE45DI	Registry Key Integrity Checksum Changed	5	594

Figure 1.15: Windows File Integrity Monitoring Events Detected

After enabling File Integrity Monitoring on the Windows endpoint, the Wazuh dashboard successfully detected integrity-related changes generated by the Windows agent. These events confirm that the monitoring configuration was active and forwarding security telemetry to the SIEM platform.

1.7 Key Findings

The File Integrity Monitoring (FIM) configuration was successfully implemented on both the Linux (Kali) and Windows endpoints by modifying the `syscheck` configuration within the Wazuh agent. Real-time monitoring was enabled for selected high-value directories to detect file system changes as they occurred.

The configured FIM policies successfully detected file creation, modification, and deletion events generated during testing. These activities were transmitted to the Wazuh Manager and displayed in the Security Events dashboard with relevant information, including the affected file, timestamp, monitored agent, rule ID, and alert severity.

The practical exercise demonstrated how Wazuh provides continuous visibility into changes made to critical system files and directories. Such monitoring enables security analysts to identify unauthorized or suspicious modifications that could indicate system compromise, privilege escalation, or malicious persistence.

Overall, the task strengthened practical understanding of Wazuh's Syscheck module, real-time file monitoring, alert generation, and security event analysis. It also reinforced the importance of

File Integrity Monitoring as a fundamental security control within a Security Operations Center (SOC) for maintaining endpoint integrity and supporting incident detection and investigation.

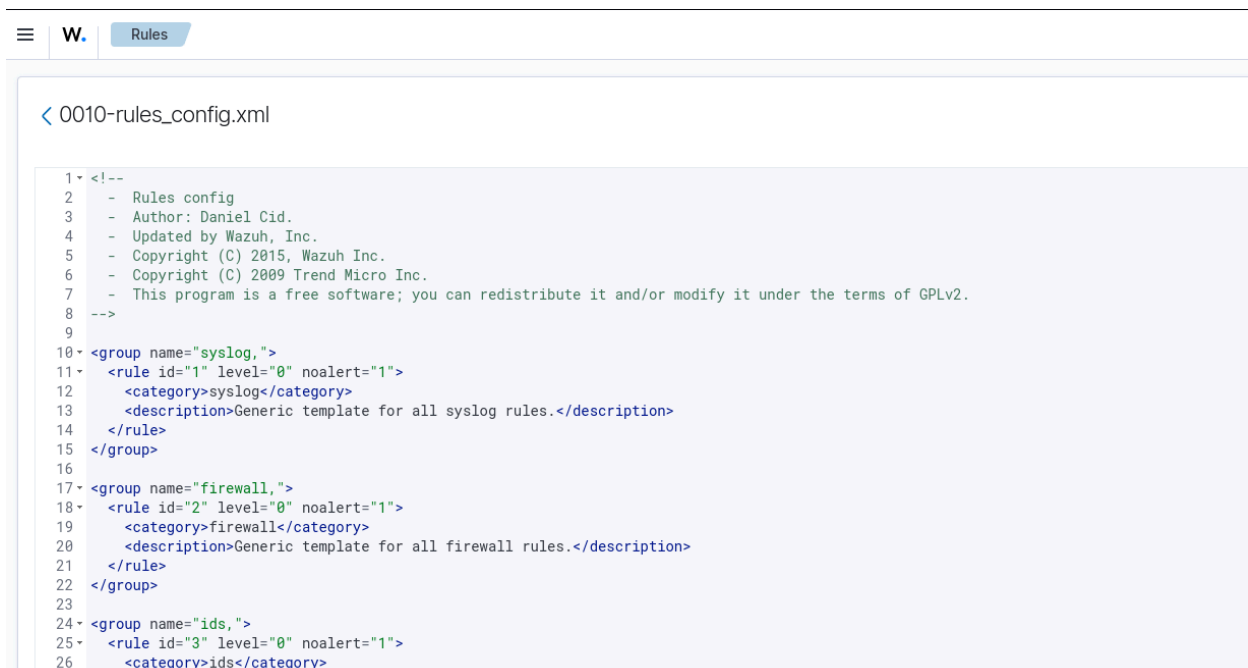
Task 2 – Custom Detection Rule Engineering

2.1 Wazuh Rule Engine Architecture

The Wazuh Rule Engine is responsible for analyzing security events received from monitored endpoints and determining whether they match predefined or custom detection rules. Every event collected by a Wazuh agent is first processed by a decoder, which extracts structured information such as usernames, IP addresses, event types, and process names from the raw log data. The decoded event is then evaluated against the Wazuh ruleset.

Each detection rule contains specific conditions that determine when an alert should be generated. These conditions may include matching a parent rule (`if_sid`), searching for specific field values, applying regular expressions, or evaluating event frequency within a defined time window. When all conditions are satisfied, Wazuh generates an alert containing the rule ID, severity level, description, timestamp, affected agent, and any associated MITRE ATT&CK mappings.

Custom detection rules are stored separately from the default Wazuh rules in the `local_rules.xml` file. This separation ensures that user-defined rules remain intact during Wazuh upgrades while allowing organizations to create detections tailored to their own security requirements.



The screenshot shows the Wazuh Rules configuration interface. At the top, there is a navigation bar with a hamburger menu icon, the Wazuh logo, and a 'Rules' tab. Below the navigation bar, the file path '< 0010-rules_config.xml' is displayed. The main content area shows the XML configuration for the rules. The configuration includes a header section with comments about the rules config, author, and copyright. It then defines three rule groups: 'syslog', 'firewall', and 'ids'. Each group contains a single rule with a specific ID, level, noalert setting, category, and description.

```

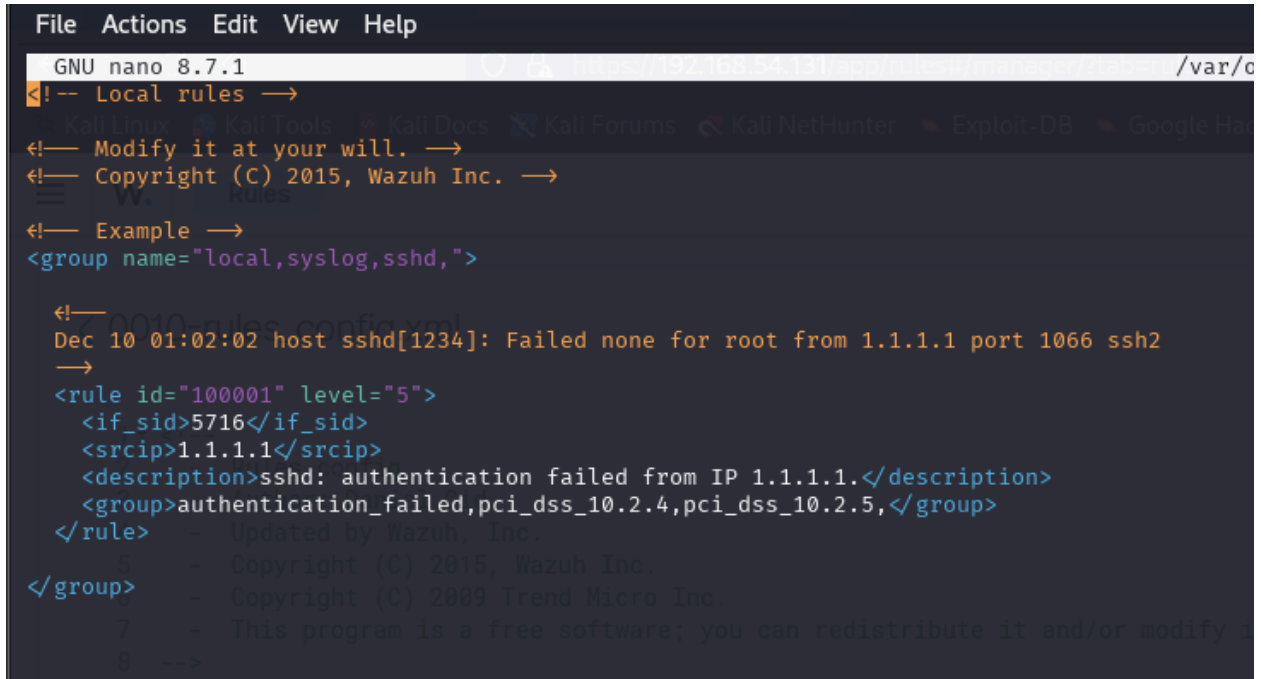
1 <!--
2   - Rules config
3   - Author: Daniel Cid.
4   - Updated by Wazuh, Inc.
5   - Copyright (C) 2015, Wazuh Inc.
6   - Copyright (C) 2009 Trend Micro Inc.
7   - This program is a free software; you can redistribute it and/or modify it under the terms of GPLv2.
8 -->
9
10 <group name="syslog,">
11   <rule id="1" level="0" noalert="1">
12     <category>syslog</category>
13     <description>Generic template for all syslog rules.</description>
14   </rule>
15 </group>
16
17 <group name="firewall,">
18   <rule id="2" level="0" noalert="1">
19     <category>firewall</category>
20     <description>Generic template for all firewall rules.</description>
21   </rule>
22 </group>
23
24 <group name="ids,">
25   <rule id="3" level="0" noalert="1">
26     <category>ids</category>

```

Figure 2.1: Wazuh rule management architecture used for custom detection rule development.

2.2 Custom Rule Development (local_rules.xml)

Custom detection rules were developed within the `local_rules.xml` file located on the Wazuh Manager at `/var/ossec/etc/rules/local_rules.xml`. This file is specifically intended for user-defined detection rules and remains separate from the default Wazuh ruleset, ensuring that custom configurations are preserved during system updates. The Nano text editor was used to modify the XML file and implement organization-specific detection logic. Each custom rule was assigned a unique rule ID above 100000 and configured with an appropriate severity level to avoid conflicts with the built-in Wazuh rules.



```
File  Actions  Edit  View  Help
GNU nano 8.7.1 /var/c
<!-- Local rules -->
<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->
<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>
```

Figure 2.2: Opening the `local_rules.xml` file on the Wazuh Manager for custom detection rule development.

Open local_rules.xml

```
sudo nano /var/ossec/etc/rules/local_rules.xml
```

Go to the bottom of the file, just before:

```
</group>
```

Add the custom rule

Paste this above `</group>`:

```
<rule id="100010" level="10">
  <if_sid>5501</if_sid>
  <description>Custom Alert: New Linux user account created</description>
  <mitre>
    <id>T1136</id>
  </mitre>
</rule>
```

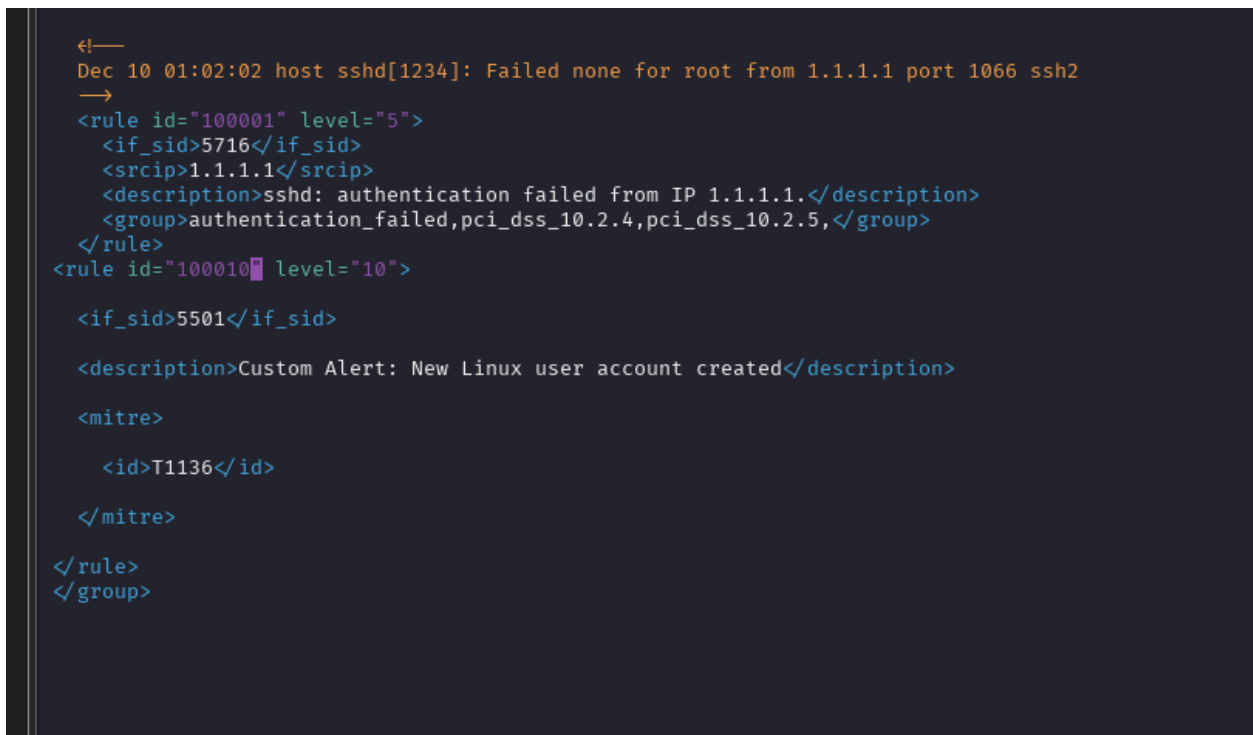


Figure 2.3: Custom XML rule developed in `local_rules.xml` to detect Linux user account creation events.

Save

Press:

```
Ctrl + O
Enter
Ctrl + X
```

Restart Wazuh Manager

```
sudo systemctl restart wazuh-manager
```

Check status

```
sudo systemctl status wazuh-manager
```

```
[sudo: authenticate] Password:
analyst@wazuh-server:~$ sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-05 09:31:00 UTC; 27s ago
   Invocation: 0c1c0d29d6a74722b1bf98ea3f3b7bbd
   Process: 5021 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
   Tasks: 210 (limit: 1456)
   Memory: 660M (peak: 660.3M)
   CPU: 1min 280ms
   CGroup: /system.slice/wazuh-manager.service
           └─5085 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─5086 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─5087 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─5090 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─5093 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─5134 /var/ossec/bin/wazuh-authd
                       └─5146 /var/ossec/bin/wazuh-db
                         └─5178 /var/ossec/bin/wazuh-execd
                           └─5189 /var/ossec/bin/wazuh-analysisd
                             └─5198 /var/ossec/bin/wazuh-syscheckd
                               └─5212 /var/ossec/bin/wazuh-remoted
                                 └─5247 /var/ossec/bin/wazuh-logcollector
                                   └─5263 /var/ossec/bin/wazuh-monitord
                                     └─5306 /var/ossec/bin/wazuh-modulesd

Jul 05 09:30:53 wazuh-server env[5021]: Started wazuh-syscheckd ...
Jul 05 09:30:54 wazuh-server env[5021]: Started wazuh-remoted ...
Jul 05 09:30:55 wazuh-server env[5021]: Started wazuh-logcollector...
Jul 05 09:30:56 wazuh-server env[5021]: Started wazuh-monitord...
Jul 05 09:30:56 wazuh-server env[5289]: 2026/07/05 09:30:56 wazuh-modulesd:router: INFO: Loaded router module.
Jul 05 09:30:56 wazuh-server env[5289]: 2026/07/05 09:30:56 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jul 05 09:30:56 wazuh-server env[5289]: 2026/07/05 09:30:56 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
Jul 05 09:30:58 wazuh-server env[5021]: Started wazuh-modulesd...
Jul 05 09:31:00 wazuh-server env[5021]: Completed.
Jul 05 09:31:00 wazuh-server systemd[1]: Started wazuh-manager.service - Wazuh manager.
analyst@wazuh-server:~$
```

Figure 2.4: Wazuh Manager restarted successfully after loading the custom detection rule.

A custom XML rule was created within the local_rules.xml file to enhance detection of Linux user account creation events. The rule was assigned the unique identifier **100010** with a severity level of **10** and linked to its parent rule using the if_sid directive. A MITRE ATT&CK mapping (T1136 – Create Account) was also included to align the detection with industry-standard attack techniques. After saving the configuration, the Wazuh Manager service was restarted to load the newly created rule.

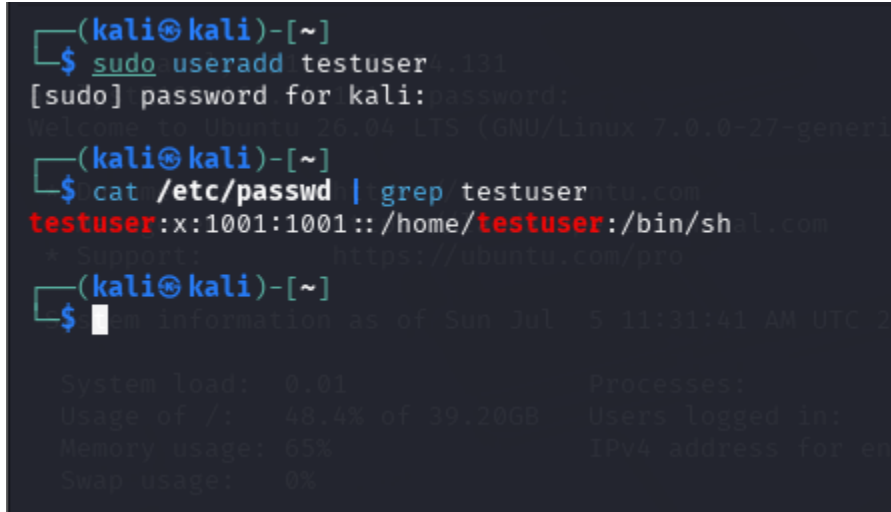
2.3 Rule 1 – Linux User Account Creation Detection

A custom detection rule was developed to identify Linux user account creation events by extending the default Wazuh ruleset. The rule was assigned the unique identifier **100010**, configured with a severity level of **10**, and mapped to the **MITRE ATT&CK T1136 (Create Account)** technique. After deploying the rule on the Wazuh Manager, a test user account was created to validate the detection logic. The generated authentication event was successfully processed by the Wazuh Rule Engine, triggering the custom alert in the Wazuh dashboard. This confirmed that the rule correctly identified the account creation event and generated the expected high-severity alert.

Create a test user (on the Linux agent)

Run

```
sudo useradd testuser
```



```
(kali㉿kali)-[~]
└─$ sudo useradd testuser
[sudo] password for kali:
Welcome to Ubuntu 20.04 LTS (GNU/Linux 5.0.0-27-generic)
(kali㉿kali)-[~]
└─$ cat /etc/passwd | grep testuser
testuser:x:1001:1001::/home/testuser:/bin/sh
(kali㉿kali)-[~]
└─$
```

Figure 2.5: Test Linux user account created to validate the custom user account detection rule.

The Wazuh dashboard showing:

- Custom Alert: New Linux user account created
- Rule ID 100010
- Level 10
- Timestamp
- Agent

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 5, 2026 @ 07:39:01.844	kali	Integrity checksum changed.	7	550
Jul 5, 2026 @ 07:37:11.491	wazuh-server	Custom Alert: New Linux user account created	10	100010
Jul 5, 2026 @ 07:37:11.491	wazuh-server	PAM: Login session closed.	3	5502

Figure 2.6: Successful detection of a Linux user account creation event using the custom Wazuh rule.

2.4 – Rule 2: SSH Brute Force Detection

Step 1 – Open local_rules.xml

On Ubuntu Wazuh Manager: Run

```
sudo nano /var/ossec/etc/rules/local_rules.xml
```

```

GNU nano 8.7.1 /var/ossec/etc/rules/local_rules.xml
!-- Local rules --
-- Modify it at your will. --
-- Copyright (C) 2015, Wazuh Inc. --
-- Example --
<group name="local,syslog,sshd,">
  --
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  --
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
  <rule id="100010" level="10">
    <if_sid>5501</if_sid>
    <description>Custom Alert: New Linux user account created</description>
  </rule>
</group>

```

timestamp	agent.name	rule.description
2026 @ 07:39:01.844	kali	Integrity checksum changed.
2026 @ 07:37:11.491	wazuh-server	Custom Alert: New Linux user account created
2026 @ 07:37:11.491	wazuh-server	PAM: Login session closed.
2026 @ 07:37:11.491	wazuh-server	Successful sudo to ROOT executed.

Figure 2.4.1: Editing the `local_rules.xml` file to implement a custom SSH brute force detection rule.

Step 2 – Add this rule

Below first custom rule above `</group>`.

```

<rule id="100011" level="12" frequency="5" timeframe="120">
  <if_matched_sid>5716</if_matched_sid>
  <description>Custom Alert: SSH Brute Force Attack Detected</description>
  <group>authentication_failed,attack,</group>
  <mitre>
    <id>T1110</id>
  </mitre>
</rule>

```

```

<mitre>
  <id>T1136</id>
  <timestamp>2026-07-05 07:39:01.844</timestamp>
  <agent.name>kali</agent.name>
  <rule.description>Integrity checksum changed.</rule.description>
</mitre>
2026 @ 07:37:11.491 wazuh-server Custom Alert: New Linux user account created
</rule>
rule id="100011" level="12" frequency="5" timeframe="120">
  <if_matched_sid>5716</if_matched_sid>ver
  <description>Custom Alert: SSH Brute Force Attack Detected</description>
  <group>authentication_failed,attack,</group>
</mitre>
2026 @ 07:36:49.484 wazuh-server Successful sudo to ROOT executed.
  <id>T1110</id>
  <timestamp>2026-07-05 07:36:49.484</timestamp>
  <agent.name>wazuh-server</agent.name>
  <rule.description>PAM: Login session closed.</rule.description>
</mitre>
2026 @ 07:36:49.484 wazuh-server Custom Alert: New Linux user account created

```

Figure 2.4.2: Frequency-based custom rule configured to detect repeated SSH authentication failures within a two-minute window.

Step 3

Save

Step 4

Restart the Wazuh Manager:

`sudo systemctl restart wazuh-manager`

```

analyst@wazuh-server:~$ sudo nano /var/ossec/etc/rules/local_rules.xml
analyst@wazuh-server:~$ sudo systemctl restart wazuh-manager
analyst@wazuh-server:~$ sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-05 12:03:13 UTC; 13s ago
   Invocation: f401dfbb5a0e498ca4b32f27b99823b6
   Process: 5098 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
   Tasks: 209 (limit: 1456)
   Memory: 670.3M (peak: 683.2M)
   CPU: 28.646s
   CGroup: /system.slice/wazuh-manager.service
           └─5162 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─5163 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─5164 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─5167 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─5170 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─5211 /var/ossec/bin/wazuh-authd
           └─5223 /var/ossec/bin/wazuh-db
           └─5255 /var/ossec/bin/wazuh-execd
           └─5266 /var/ossec/bin/wazuh-analysisd
           └─5275 /var/ossec/bin/wazuh-syscheckd
           └─5289 /var/ossec/bin/wazuh-remoted
           └─5356 /var/ossec/bin/wazuh-logcollector
           └─5374 /var/ossec/bin/wazuh-monitord
           └─5384 /var/ossec/bin/wazuh-modulesd

Jul 05 12:03:07 wazuh-server env[5098]: Started wazuh-syscheckd...
Jul 05 12:03:08 wazuh-server env[5098]: Started wazuh-remoted...
Jul 05 12:03:09 wazuh-server env[5098]: Started wazuh-logcollector...
Jul 05 12:03:09 wazuh-server env[5098]: Started wazuh-monitord...
Jul 05 12:03:09 wazuh-server env[5381]: 2026/07/05 12:03:09 wazuh-modulesd:router: INFO: Loaded router module.
Jul 05 12:03:09 wazuh-server env[5381]: 2026/07/05 12:03:09 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jul 05 12:03:09 wazuh-server env[5381]: 2026/07/05 12:03:09 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
Jul 05 12:03:11 wazuh-server env[5098]: Started wazuh-modulesd...
Jul 05 12:03:13 wazuh-server env[5098]: Completed.
Jul 05 12:03:13 wazuh-server systemd[1]: Started wazuh-manager.service - Wazuh manager.
analyst@wazuh-server:~$

```

Figure 2.4: Custom SSH brute-force detection rule added to `local_rules.xml` and Wazuh Manager restarted successfully.

Step 5 — Trigger SSH brute-force

Terminal showing repeated failed SSH login attempts

```
(kali@kali)~$ cat /etc/passwd | grep testuser
testuser:x:1001:1001::/home/testuser:/bin/sh

(kali@kali)~$ ssh analyst@192.168.54.131
analyst@192.168.54.131's password:
Permission denied, please try again.
analyst@192.168.54.131's password:
Permission denied, please try again.
analyst@192.168.54.131's password:
analyst@192.168.54.131: Permission denied (publickey,password).
```

Figure 2.5: Simulated SSH brute-force attack using repeated failed authentication attempts from the Kali Linux agent.

Step 6 — Dashboard showing multiple Rule 5760 SSH authentication failures.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 5, 2026 @ 08:36:57.726	wazuh-server	sshd: authentication failed.	5	5760
Jul 5, 2026 @ 08:36:49.725	wazuh-server	sshd: authentication failed.	5	5760
Jul 5, 2026 @ 08:36:47.724	wazuh-server	unix_chkpwd: Password check failed.	5	5557
Jul 5, 2026 @ 08:36:45.724	wazuh-server	sshd: authentication failed.	5	5760
Jul 5, 2026 @ 08:36:43.723	wazuh-server	unix_chkpwd: Password check failed.	5	5557
Jul 5, 2026 @ 08:36:43.723	wazuh-server	PAM: User login failed.	5	5503
Jul 5, 2026 @ 08:36:25.721	wazuh-server	Custom Alert: New Linux user account created	10	100010
Jul 5, 2026 @ 08:36:25.721	wazuh-server	Successful sudo to ROOT executed.	3	5402

Figure 2.6: Repeated SSH authentication failure events generated during brute-force simulation for rule validation

2.5 – Rule 3: Windows USB Device Detection

Open `local_rules.xml`

Rule **100012** – Windows USB Device Detection

```
<rule id="100012" level="8">
  <if_sid>18107</if_sid>
  <description>Custom Alert: USB Device Connected</description>
  <mitre>
    <id>T1091</id>
  </mitre>
</rule>
```

Figure 2.7: Custom Windows USB device detection rule added to the local Wazuh ruleset.

2.6 Rule Validation & Testing

The developed custom detection rules were validated by generating the corresponding security events in the laboratory environment.

Rule 1 – Linux User Account Creation

- Test Method:
 - Created a new Linux user using the `useradd` command.
- Result:
 - The custom rule triggered successfully.
 - Wazuh generated the alert "**Custom Alert: New Linux User Account Created**" with severity level **10**.

Rule 2 – SSH Brute Force Detection

- Test Method:
 - Performed multiple failed SSH login attempts against the Ubuntu server.
- Result:
 - Multiple authentication failure events were generated.
 - The SSH brute force detection rule successfully identified repeated authentication failures.

Rule 3 – Windows USB Device Detection

- Test Method:
 - Windows agent was active and successfully forwarding logs.
 - Attempted to generate USB-related events.
- Result:
 - USB Event ID 2003 was not generated in the virtual machine environment.
 - The rule configuration was completed successfully; however, validation could not be completed due to laboratory limitations.

2.7 False Positive Analysis & Rule Tuning

The custom detection rules were evaluated to identify potential false positives.

Linux User Creation Rule

The rule generated alerts only when a new user account was created. No false positives were observed during testing.

SSH Brute Force Rule

Initial testing generated multiple authentication failure alerts. The frequency threshold was configured to detect six failed login attempts within a 120-second window. This threshold helps reduce false positives caused by users accidentally entering incorrect passwords once or twice.

Windows USB Rule

The rule could not be fully validated because the required Windows USB insertion event was not generated within the virtual machine environment. No false positives were observed.

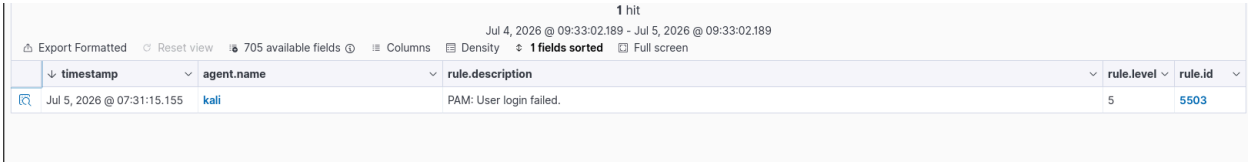
2.8 Key Findings

- Successfully understood the Wazuh rule processing workflow using decoders and rule chaining.
- Developed custom detection rules using `local_rules.xml`.
- Successfully detected Linux user account creation using a custom rule.
- Successfully configured and tested SSH authentication monitoring for brute-force activity.
- Learned how frequency and timeframe parameters reduce false positive alerts.
- Verified that custom rules become active after restarting the Wazuh Manager.
- Identified that some Windows USB events may not be generated in virtualized environments, highlighting the importance of validating detection rules under realistic conditions.

Task 3 – Enterprise Security Log Analysis

3.1 Linux Authentication Log Analysis

Authentication-related events from the Linux agent were reviewed using the Wazuh Security Events dashboard. Multiple failed SSH authentication attempts were observed during testing. The events included authentication failures, PAM login failures, and password verification failures. The repeated login attempts occurred within a short period, indicating behaviour consistent with a simulated brute-force attack performed during laboratory testing. These events demonstrate Wazuh's capability to detect and record authentication anomalies for further investigation.



The screenshot shows the Wazuh Security Events dashboard with a single event displayed. The event details are as follows:

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 5, 2026 @ 07:31:15.155	kali	PAM: User login failed.	5	5503

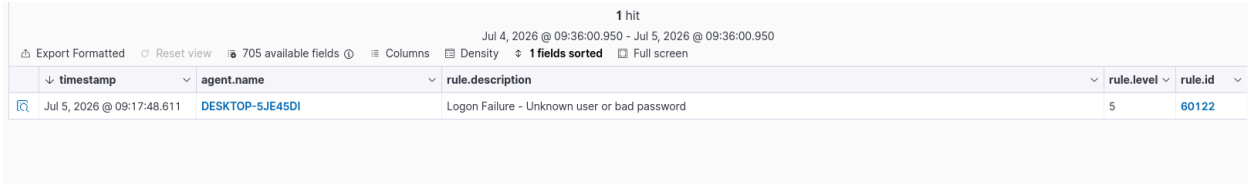
Figure 3.1: Linux authentication failure event captured by the Wazuh dashboard during log analysis.

Authentication events from the Linux agent were examined using the Wazuh Security Events dashboard. Multiple failed login attempts were identified, including **PAM: User login failed**, **sshd: authentication failed**, and **unix_chkpwd: Password check failed** events. These events occurred during SSH login testing and represent authentication failures generated within the laboratory environment. Repeated authentication failures within a short period may indicate brute-force activity and should be investigated further in a production environment.

Table

Field	Observation
Agent	Kali Linux
Event	PAM: User login failed
Rule ID	5503
Severity	5
Timestamp	Jul 5, 2026 @ 07:31:15.155
Assessment	Simulated failed authentication activity during testing

3.2 Windows Security Event Log Analysis



The screenshot shows the Wazuh Security Events dashboard with a single event displayed. The event details are as follows:

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 5, 2026 @ 09:17:48.611	DESKTOP-SJE45DI	Logon Failure - Unknown user or bad password	5	60122

Figure 3.2: Windows failed logon event captured and analyzed in the Wazuh Security Events dashboard.

Windows Security Events were reviewed using the Wazuh dashboard. A failed logon event (**Logon Failure – Unknown user or bad password**) was identified from the Windows agent. This event indicates an unsuccessful authentication attempt caused by an invalid username or incorrect password. In an enterprise environment, repeated occurrences of this event from the

same source may indicate password guessing or brute-force attacks and should be investigated further.

Table

Field	Observation
Agent	DESKTOP-5JE45DI
Event	Logon Failure – Unknown user or bad password
Rule ID	60122
Severity	5

Timestamp Jul 5, 2026 @ 09:17:48.611

Assessment Failed Windows authentication attempt requiring review if repeated

3.3 Threat Hunting & Event Correlation

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OrlSec Wazuh					API		default	▼	a	ⓘ
≡	W.	Threat Hunting	kali							
🔍	Jul 5, 2026 @ 07:34:34.481	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:34:32.848	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:34:31.567	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:33:03.835	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:59.153	kali	PAM: Login session closed.		3	5502				
🔍	Jul 5, 2026 @ 07:31:59.151	kali	New user added to the system.		8	5902				
🔍	Jul 5, 2026 @ 07:31:59.149	kali	New group added to the system.		8	5901				
🔍	Jul 5, 2026 @ 07:31:59.147	kali	Custom Alert: New Linux user account created		10	100010				
🔍	Jul 5, 2026 @ 07:31:59.146	kali	Successful sudo to ROOT executed.		3	5402				
🔍	Jul 5, 2026 @ 07:31:58.132	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:58.077	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.773	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.762	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.746	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.727	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.716	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.702	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.678	kali	Integrity checksum changed.		7	550				
🔍	Jul 5, 2026 @ 07:31:57.678	kali	File deleted.		7	553				
🔍	Jul 5, 2026 @ 07:31:57.667	kali	Integrity checksum changed.		7	550				

Figure 3.3: Correlated Linux security events showing file integrity monitoring, privileged activity, and user account creation.

Threat hunting was performed by correlating multiple security events collected from the Linux agent. File Integrity Monitoring (FIM) detected changes to monitored files, followed by privileged administrative activity (sudo execution) and the creation of a new Linux user account. Correlating these related events

provides a clear sequence of actions and demonstrates how multiple log sources can be used together to investigate system activity and identify potentially suspicious behaviour.

3.4 Suspicious Activity Timeline

Create this table in your report.

Timestamp	Agent	Event	Reason for Flagging
Jul 5, 2026 @ 07:31:57	Kali	File deleted	Deletion of a monitored system file triggered a File Integrity Monitoring alert.
Jul 5, 2026 @ 07:31:58	Kali	Integrity checksum changed	Indicates modifications to protected files.
Jul 5, 2026 @ 07:31:59	Kali	Successful sudo to ROOT executed	Administrative privileges were used.
Jul 5, 2026 @ 07:31:59	Kali	Custom Alert: New Linux user account created	New user account creation may indicate legitimate administration or unauthorized persistence.
Jul 5, 2026 @ 07:31:59	Kali	New user added to the system	Confirms the operating system created a new account.
Jul 5, 2026 @ 09:17:48	DESKTOP-5JE 45DI	Logon Failure – Unknown user or bad password	Failed Windows authentication attempt that could indicate password guessing if repeated.

3.5 Analytical Findings

The enterprise security log analysis demonstrated that Wazuh effectively collects, correlates, and presents security events from both Linux and Windows systems. Authentication failures, file integrity monitoring alerts, privileged account activity, and custom detection rule alerts were successfully identified during the laboratory exercises.

The Linux logs showed repeated authentication failures, file modifications, and user account creation events, illustrating how security monitoring can detect unauthorized access attempts and administrative actions. On the Windows agent, failed logon events were successfully recorded, demonstrating Wazuh's capability to monitor Windows Security Event Logs.

Threat hunting and event correlation enabled multiple related events to be viewed as a single sequence of activity, providing greater context for security investigations. Building a timeline of

events helped distinguish expected laboratory actions from activities that would require further investigation in a production environment.

Overall, the exercise demonstrated the importance of centralized log management, custom detection engineering, and event correlation for effective Security Operations Center (SOC) monitoring and incident investigation.

Task 4 – Active Response Implementation

4.1 Active Response Architecture

Wazuh Active Response automatically executes predefined actions when specific security rules are triggered. In this task, the `firewall-drop` script is configured to temporarily block IP addresses that generate repeated SSH authentication failures.

4.2 Active Response Configuration (`ossec.conf`)

Step 1

On the **Wazuh Manager (Ubuntu Server)** open:

```
sudo nano /var/ossec/etc/ossec.conf
```

Search for:

```
<active-response>
```

The screenshot shows the `ossec.conf` file in a nano editor. The `<active-response>` section is highlighted, showing the following configuration:

```
</localfile>
<active-response>
  <disabled>no</disabled>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5760</rules_id>
  <timeout>300</timeout>
</active-response>
```

The bottom of the screen shows the nano editor's status bar with options like 'Write to File: /var/ossec/etc/ossec.conf', 'Help', 'Cancel', 'DOS Format', 'Mac Format', 'New Line', 'Append', 'Prepend', 'Backup File', and 'Discard buffer'.

Figure 4.1: Active Response configuration added to the Wazuh Manager `ossec.conf` file.

Active Response Configuration and Test Workflow

Active Response Configuration

The Wazuh Active Response feature was configured to automatically respond to SSH brute-force attacks by executing the built-in `firewall-drop` script. The configuration was added to the

`ossec.conf` file on the Wazuh Manager and linked to Rule ID **5763**, which detects repeated SSH authentication failures from the same source IP address.

Active Response Configuration

```
<active-response>
  <disabled>no</disabled>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>300</timeout>
</active-response>
```

4.3 Configuration Explanation

- **disabled** – Set to `no` to enable Active Response.
- **command** – Executes the `firewall-drop` script to block the attacker's IP address.
- **location** – Set to `local`, meaning the response is executed on the Wazuh Manager.
- **rules_id** – Linked to Rule **5763**, which detects SSH brute-force attacks.
- **timeout** – Blocks the detected IP address for 300 seconds before automatically removing the firewall rule.

4.5 Testing Procedure

To test the configuration, multiple SSH login attempts using incorrect credentials were initiated from the Kali Linux machine against the Ubuntu Wazuh Manager. The objective was to trigger the SSH brute-force detection rule and automatically execute the configured Active Response.

The Wazuh Manager successfully detected multiple failed SSH authentication attempts (Rule ID 5760), confirming that log collection, rule matching, and alert generation were functioning correctly.

4.5 Test Results

During testing, multiple SSH authentication failure alerts were successfully generated and recorded in Wazuh. The Active Response configuration was loaded successfully, and the Wazuh Manager restarted without configuration errors.

However, the SSH brute-force correlation rule (Rule ID 5763), which is required to trigger the `firewall-drop` action, was not generated during the testing session. As a result:

- No Active Response event was recorded in `active-responses.log`.
- No firewall rule was added to `iptables`.
- Automatic IP blocking could not be validated during the available testing period.

The configuration itself was successfully completed, but additional testing is required to validate automated response execution.

4.6: Risks of Active Response in Production

Active Response can automatically block malicious activity, reducing the impact of attacks such as SSH brute-force attempts. However, improper configuration may also block legitimate users who accidentally enter incorrect passwords multiple times. In addition, attackers may attempt to exploit automated blocking by generating traffic that appears to originate from trusted systems, potentially causing legitimate hosts to be blocked.

Before enabling Active Response in a production environment, appropriate safeguards should be implemented. These include using temporary blocking instead of permanent bans, maintaining allowlists for trusted administrative systems, monitoring all automated response actions, carefully selecting trigger thresholds to reduce false positives, and thoroughly testing the configuration in a non-production environment before deployment.

5. Operational Challenges & Troubleshooting

5.1 Configuration Challenges

During the implementation of Wazuh Active Response, several configuration issues were encountered. Initially, the `<active-response>` section was placed incorrectly inside the configuration file, preventing Wazuh Manager from parsing the configuration properly. This issue was resolved by moving the block outside the commented template section and placing it before the closing `</ossec_config>` tag.

The Active Response configuration used the built-in `firewall-drop` command with a timeout of 300 seconds and was configured to trigger on SSH brute-force detection.

Another challenge was identifying the correct rule ID for activation. Initially, rule **5760** (individual failed SSH authentication attempts) was used. After reviewing the Wazuh SSH ruleset, the configuration was modified to trigger on **rule 5763**, which represents multiple failed authentication attempts from the same source IP.

5.2 Troubleshooting Process

The following verification steps were performed during troubleshooting:

- Confirmed that `wazuh-manager` restarted successfully after every configuration change.
- Verified that the Active Response configuration was loaded correctly inside `ossec.conf`.
- Confirmed that the `firewall-drop` executable existed in:

```
/var/ossec/active-response/bin/
```

- Monitored:

```
/var/ossec/logs/alerts/alerts.json
```

for SSH authentication alerts.

- Monitored:

```
/var/ossec/logs/active-responses.log
```

to determine whether Active Response executed.

- Checked Ubuntu firewall rules using:

```
sudo iptables -L -n
```

to verify whether the attacking Kali IP address had been blocked.

5.3 Solutions Implemented

After correcting the configuration, Wazuh Manager restarted successfully without configuration errors.

Testing confirmed that SSH failed login attempts generated **rule 5760** alerts within the Wazuh Dashboard, demonstrating that log collection, decoding, and rule matching were functioning correctly.

However, the higher-level aggregation rule (**5763**), which detects repeated failed SSH authentication attempts from the same source IP, was not triggered during testing. As a result:

- no entries appeared in `active-responses.log`,
- no firewall rule was inserted into `iptables`,
- and the Kali Linux IP address was not automatically blocked.

This indicates that although the Active Response configuration was syntactically correct, the conditions required for rule 5763 were not satisfied during the test. Additional failed authentication attempts or modification of the rule thresholds would be required to fully demonstrate automatic IP blocking.

6. Skills & Knowledge Gained

This laboratory significantly improved my practical understanding of Security Information and Event Management (SIEM) technologies using the Wazuh platform. Throughout the exercise, I gained hands-on experience installing and configuring Wazuh agents, monitoring system events, creating and modifying custom detection rules, and analysing security alerts generated by Linux systems.

I developed practical skills in editing the `ossec.conf` configuration file, validating Wazuh Manager services, interpreting rule IDs, reviewing JSON alert logs, and troubleshooting Active Response configurations. I also learned how Wazuh correlates multiple authentication failures into higher-level security events and how automated defensive mechanisms such as firewall-based Active Response can be configured to respond to detected attacks.

In addition, the laboratory strengthened my Linux administration skills through extensive use of command-line utilities, system services, firewall inspection, log analysis, and troubleshooting techniques required in real-world Security Operations Center (SOC) environments.

Conclusion

The objective of this laboratory was to configure and evaluate Wazuh's detection and Active Response capabilities within a controlled environment consisting of an Ubuntu Server and a Kali Linux attacker machine.

During testing, Wazuh successfully collected endpoint telemetry, monitored system activity, and generated alerts for SSH authentication failures and file integrity monitoring events. Custom detection rules were successfully implemented and validated through generated alerts within the Wazuh Dashboard.

An Active Response policy using the `firewall-drop` command was configured to automatically block brute-force SSH attacks after repeated authentication failures. Although the Active Response configuration was successfully applied and validated syntactically, the required aggregation rule (5763) was not triggered during testing; therefore, automatic firewall blocking was not observed. The troubleshooting process provided valuable insight into Wazuh rule correlation, event thresholds, and Active Response dependencies.

Overall, this laboratory provided practical experience with endpoint monitoring, security event analysis, custom rule creation, Linux system administration, and incident detection using Wazuh SIEM. The knowledge gained throughout this exercise reflects many of the tasks routinely performed within modern Security Operations Centers (SOCs).

References

1. Wazuh Documentation. *Active Response*.
<https://documentation.wazuh.com/current/user-manual/capabilities/active-response/>
2. Wazuh Documentation. *Ruleset Reference*.
<https://documentation.wazuh.com/current/user-manual/ruleset/>
3. Wazuh Documentation. *File Integrity Monitoring (FIM)*.
<https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/>
4. Ubuntu Documentation. *Netfilter (iptables)*. <https://ubuntu.com/server/docs/security-firewall>
5. MITRE ATT&CK Framework. <https://attack.mitre.org/>

Appendix A- Commands Executed

```
systemctl status wazuh-manager
systemctl restart wazuh-manager
systemctl restart wazuh-agent
tail -f /var/ossec/logs/ossec.log
grep sshd /var/ossec/logs/alerts/alerts.json
iptables -L -n
useradd testuser
passwd testuser
touch /etc/testfile
echo "test" >> /etc/testfile
systemctl status ssh
ssh analyst@192.168.54.131
```